

Philips Hue – Interface Help

JUNG Visu Pro Process Connection – InterfaceHelp – Version 1

1. Overview

This process connection links JVP to a Philips Hue Bridge on the local network (CLIP v1 REST API) and exposes lights, rooms/groups, scenes, sensors, and automations (Hue rules) as datapoints. Optionally, the presence status of the Hue app can also be queried (geofencing, CLIP v2/HTTPS).

For performance reasons, states are only queried from the bridge once at startup or on request (a Hue installation rarely changes). Switching commands from JVP are independent of this and always take effect immediately. Each light can additionally be switched to automatic polling individually.

2. Setup

2.1 Find and enter the bridge IP

In the Philips Hue app: Settings → Hue Bridges → select bridge → IP address. Enter this IP under **Configuration** → **Bridge IP address**. Optionally enable **Auto-update bridge IP** so the interface takes care of a changed IP itself on connection loss (see section 3.5).

2.2 Pair with the bridge

Set **Start pairing** to 1, then press the round link button on the bridge within 5 minutes. **Pairing status** changes to "Paired" once successful, and the **API key** is filled in automatically (can alternatively be entered manually if a key from another installation already exists). On "Timeout", repeat and press the link button a bit more promptly.

2.3 Find the Hue IDs

Set **List inventory** to 1. Under Status, five fields then contain a tab-separated list of Hue ID and name (Inventory: lights/groups/scenes/sensors/automations), sorted by ID – paste directly into Excel or use as a cheat sheet for step 2.4.

2.4 Create instances

Each category has its own main folder. Create a new instance for each device inside it and enter the Hue ID (the name is then reloaded automatically):

Main folder	Instance	For	ID field
Lights	Light	each individual light	Hue light ID
Rooms/Groups	Group	each room/group	Hue group ID
Scenes	Scene	each scene	Hue scene ID
Sensors	Sensor	each sensor/button/contact	Hue sensor ID
Automations	Automation	each Hue rule	Hue rule ID

Alternative – auto-assign IDs: Create empty instances in the desired order, then set **Auto-assign IDs** to 1 – instance 1 in each main folder gets the first ID found, instance 2 the second, etc. Only empty ID fields are filled in; can be re-run any time for newly added instances.

3. Operation

3.1 Send and receive datapoints

Every controllable value exists twice: as a **send datapoint** (no suffix, write-only – bind to the control element) and as a **receive datapoint** (suffix "(feedback)", read-only – bind to the status display). Example: "On/Off" switches, "On/Off (feedback)" shows the actual state per the bridge.

3.2 Automatic polling per light

Every light has a **Poll interval s** field, pre-filled with 10 (seconds) for new instances. 0 = no automatic polling (only at startup or on request). A different value sets the cadence individually per light.

3.3 Refresh states manually

Set **Refresh states** to 1 to pick up bridge-side changes (e.g. made via the Hue app) in JVP immediately, without waiting for the next automatic poll cycle.

3.4 Presence/geofencing (optional)

Set **Query presence (geofencing)** to "on" to query the home/away status of the Hue app. Only works if the JVP HTTP resource accepts HTTPS with the bridge's self-signed certificate. If stuck on "no one home" or errors appear under "Last error", disable the function again – the rest of the functionality is unaffected.

3.5 Auto-update the bridge IP (optional)

Hue bridges normally get their IP address via DHCP – if it changes (e.g. after a router restart), the interface would otherwise stay disconnected permanently. With **Auto-update bridge IP** (Configuration), the current IP is looked up via Philips' official cloud service discovery.meethue.com on detected connection loss and adopted into Bridge IP address automatically (requires internet access, at most one attempt every 5 minutes). Leave it off without internet access and fix the bridge's IP via a DHCP reservation in the router instead.

4. Datapoint reference

All fields are arranged in setup order (top to bottom: input first, then its result).

4.1 Configuration

Field	Access	Description
Bridge IP address	Read/Write	IP address of the Hue Bridge on the local network, e.g. 192.168.1.20
Auto-update bridge IP	Read/Write	On connection loss, automatic IP lookup via discovery.meethue.com (cloud, requires internet access)
Start pairing	Write	Write 1, then press the link button on the bridge within 5 minutes
API key (username)	Read/Write	Filled in automatically during pairing, or entered manually
List inventory	Write	Write 1: lists all lights/groups/scenes/sensors/rules with their Hue ID under Status
Auto-assign IDs	Write	Write 1: assigns the IDs found on the bridge to created instances in order
Refresh states	Write	Write 1: queries all states from the bridge once
Query presence (geofencing)	Read/Write	Queries the home/away status of the Hue app via CLIP v2 (HTTPS)
Clear debug log	Write	Write 1 to reset the debug log

4.2 Status

Field	Access	Description
Pairing status	Read	Text status of the pairing process
Connection	Read	Bridge currently reachable
Inventory: lights/groups/scenes/sensors/automations	Read	One list each (Hue ID + name), generated via List inventory
Last update	Read	Time of the last status refresh
Presence	Read	At least one device reports "home" (only with geofencing enabled)
Last error	Read	Latest error message
Debug log	Read	Rolling log of the latest messages for troubleshooting

4.3 Lights → Light

Field	Access	Description
Hue light ID	Read/Write	ID of the light on the bridge
Name	Read	Loaded automatically from the bridge
Poll interval s	Read/Write	Seconds between automatic queries, default 10s, 0 = off
On/Off	Write	Send datapoint: switches the light
On/Off (feedback)	Read	Receive datapoint: current switch state
Brightness %	Write	Send datapoint: sets the brightness
Brightness % (feedback)	Read	Receive datapoint: current brightness
Color temperature K	Write	Send datapoint: 2000 (warm white) .. 6500 (cool white)
Color temperature K (feedback)	Read	Receive datapoint: current color temperature
Hue angle	Write	Send datapoint: 0..360°, color lights only
Hue angle (feedback)	Read	Receive datapoint: current hue angle
Saturation %	Write	Send datapoint: color lights only
Saturation % (feedback)	Read	Receive datapoint: current saturation
Reachable	Read	Reachability per the bridge (if reported)
Last update	Read	Time this light was last refreshed

4.4 Rooms/Groups → Room/Group

Field	Access	Description
Hue group ID	Read/Write	ID of the group/room on the bridge
Name	Read	Loaded automatically from the bridge
On/Off	Write	Send datapoint: switches the group/room
On/Off (feedback)	Read	Receive datapoint: current switch state
Brightness %	Write	Send datapoint: sets the brightness of the group/room
Brightness % (feedback)	Read	Receive datapoint: current brightness
Any on	Read	At least one light in the group is on
All on	Read	All lights in the group are on

4.5 Scenes → Scene

Field	Access	Description
Hue scene ID	Read/Write	ID of the scene on the bridge
Target group ID	Read/Write	Room the scene is recalled in (0 = all lights); auto-filled from the bridge for room scenes
Name	Read	Loaded automatically from the bridge
Room	Read	Name of the associated room, to tell apart identically named scenes (e.g. "Nightlight") in different rooms
Activate	Write	Write 1: recalls the scene (no feedback counterpart, scenes have no persistent state)

4.6 Sensors → Sensor

Field	Access	Description
Hue sensor ID	Read/Write	ID of the sensor on the bridge
Name	Read	Loaded automatically from the bridge
Type	Read	Hue API type, see table 4.6.1
Value (text)	Read	Readable value, depends on type
Value (numeric)	Read	Temperature °C, lux, presence/contact 0/1, or button event
Contact (window/door)	Read	Only set for window contacts: 1=open, 0=closed
Battery %	Read	Battery level, if reported by the device
Last update	Read	Time of the last refresh

Table 4.6.1 – Sensor types

Device type	Hue type	Value (text)
Motion detector	ZLLPresence / ZGPPresence / CLIPPresence	"Motion detected" / "No motion"
Temperature sensor	ZLLTemperature / CLIPTemperature	e.g. "21.3 C"
Light-level sensor	ZLLLightLevel / CLIPLightLevel	e.g. "150 lx"
Button/remote/doorbell	ZLLSwitch / ZGPSwitch / ZHASwitch / CLIPSwitch	button event code
Window contact	ZHAOpenClose / CLIPOpenClose	"open" / "closed"
Daylight sensor	Daylight	"Day" / "Night"

4.7 Automations → Automation

Field	Access	Description
Hue rule ID	Read/Write	ID of the rule/automation on the bridge
Name	Read	Loaded automatically from the bridge
Enabled	Write	Send datapoint: enables/disables the automation
Enabled (feedback)	Read	Receive datapoint: current state per the bridge
Trigger count	Read	Number of times triggered per the bridge
Last triggered	Read	Time at which JVP detected the last trigger

5. Troubleshooting

Symptom	Cause/solution
Connection stays "disconnected"	Bridge IP wrong, API key empty, or network unreachable
Light doesn't respond	Hue light ID wrong/empty, or the instance folder hasn't been reloaded
Pairing fails	Link button not pressed in time – repeat
Error message about SCRIPTNAME when loading	Old interface version in the editor cache – close and reopen the editor
Bridge IP isn't auto-updated despite being enabled	No internet access for the JVP HTTP resource, or discovery.meethue.com unreachable – enter the IP manually instead